

Step 1 — A Classification Chain With `RunnableBranch` — Hints

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (the real LCEL calls, in the right order), **Advanced** (the ambiguous-ticket question this step is really testing). Read Basic first even if you've used LangChain before — it's the fastest way to spot exactly what each deeper level adds.

- [Hint 1 — What `RunnableBranch` actually needs](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — What `RunnableBranch` actually needs

Basic Version

Think of `RunnableBranch` as a stack of "if this, do that" checks, tried in order, with one final "otherwise, do this" at the end. You give it a list of pairs — a check, and what to run if that check is true — plus one last thing to run if none of the checks matched.

Things to use: - A classifier chain that reads a ticket and decides its category. - `RunnableBranch` from `langchain_core.runnables` . - Three small chains, one per category, to route to.

Intermediate Version

`RunnableBranch` is built like this: `RunnableBranch((check1, chain1), (check2, chain2), default_chain)` . Each `check` is a plain function that takes the current input (a dict) and returns `True` or `False` . `RunnableBranch` tries each check, in the order you wrote them, and runs the first chain whose check passed. If none passed, it runs `default_chain` — the one item at the end with no check attached to it.

Because each `check` just needs to be *callable*, a small named function works exactly the same way a `lambda` would — `def is_billing(inputs): return inputs["category"] == "billing"` — and it's much easier for someone else (or you, in six months) to read a stack trace that names a real function instead of `<lambda>` .

Before `RunnableBranch` can check `inputs["category"]` , something has to put `category` into the input dict in the first place — that's what `RunnableParallel` is for here:

`RunnableParallel(ticket=RunnablePassthrough(), category=classification_chain)` runs the classifier and tags the original ticket with the result, in one step, producing exactly the shape `RunnableBranch` 's checks expect.

The exact pieces: - `from langchain_core.runnables import RunnableBranch, RunnableParallel, RunnablePassthrough` - `.with_structured_output(TicketClassification)` on your classifier's model, so `category` comes back as a real, checked field, not a string you have to clean up yourself. - Named predicate functions (`is_billing` , `is_technical`), not `lambda` . - `general_chain` as the last, unpaired item in `RunnableBranch` — that's what makes it the default.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Here's the real question this step is testing, and it isn't "does the happy-path routing work." Run the ambiguous ticket from **A Real Example** ("my invoice looks wrong and the dashboard doesn't match it either") through your router. Watch which branch actually catches it.

Whatever your classifier decided, it decided with *some* amount of confidence — even if that confidence never shows up anywhere in your code right now. A `RunnableBranch` with only 3 named branches and one default has no way to express "I'm not sure" — it can only ever fully commit to one of the paths you gave it. That's not a bug in `RunnableBranch`; it's an honest limit of what a fixed-branch router can express at all.

The design question worth sitting with: should this step quietly accept that "not sure" and "general" end up looking identical downstream? Or should something about the classifier's uncertainty survive past this step, so a later part of the pipeline (Step 4's `needs_human_review` field) can actually see it? You don't have to solve this in Step 1 — but you should be able to point at your ambiguous-ticket test run and say, honestly, "here's what happens, and here's whether that's good enough yet."

Difference between Basic, Intermediate, and Advanced: Basic describes `RunnableBranch` as a stack of checks with a default. Intermediate shows the real `RunnableBranch` + `RunnableParallel` calls, in the order they need to run, and why named functions beat inline `lambda`s here. Advanced doesn't add new API calls at all — it points at the one thing a 3-branch, 1-default router can never express on its own (real uncertainty), which is exactly the gap Step 4 exists to close.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
build a classifier chain: prompt + model -> TicketClassification (category)
build 3 draft chains: billing_chain, technical_chain, general_chain

tag each ticket with its category (run the classifier alongside the ticket itself)
check the category:
    if billing -> run billing_chain
    if technical -> run technical_chain
    otherwise -> run general_chain
```

Intermediate Version

```
# models.py
from pydantic import BaseModel

class TicketClassification(BaseModel):
    category: str # "billing", "technical", or "general"
```

```
# classifier.py
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
from models import TicketClassification
from config import MODEL_NAME, MODEL_TEMPERATURE

CLASSIFY_PROMPT = ChatPromptTemplate.from_template(
    "Classify this support ticket into exactly one category: "
    "billing, technical, or general.\n\nTicket: {ticket}"
)

def build_classification_chain():
    model = ChatOpenAI(model=MODEL_NAME, temperature=MODEL_TEMPERATURE)
    return CLASSIFY_PROMPT | model.with_structured_output(TicketClassification)
```

Now write `handlers.py`'s three chains (each its own prompt + model + `StrOutputParser()`) and try wiring the `RunnableBranch` yourself before checking the Advanced version below.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

```
# router.py
from langchain_core.runnables import RunnableBranch, RunnableParallel, RunnablePassthrough
from classifier import build_classification_chain
from handlers import billing_chain, technical_chain, general_chain

def is_billing(inputs):
    return inputs["category"].category == "billing"

def is_technical(inputs):
    return inputs["category"].category == "technical"

def build_router_chain():
    classification_chain = build_classification_chain()
    tag_with_category = RunnableParallel(
        ticket=RunnablePassthrough(),
        category=classification_chain,
    )
    router = RunnableBranch(
        (is_billing, billing_chain),
        (is_technical, technical_chain),
        general_chain, # default: no check needed, this is "otherwise"
    )
    return tag_with_category | router
```

Fill in `billing_chain`, `technical_chain`, and `general_chain` in `handlers.py` yourself (each is just `ChatPromptTemplate | model | StrOutputParser()` with a category-appropriate system prompt), then compare your finished router against the [Solution](#).

Difference between Basic, Intermediate, and Advanced: Basic is the plain "if/otherwise" shape in plain English. Intermediate builds the actual classifier and its Pydantic shape, in real code. Advanced wires the classifier's output into `RunnableBranch` through `RunnableParallel`, using named predicate functions instead of inline `lambda`s — and reading `inputs["category"].category` (a `TicketClassification` object nested inside the dict, not a bare string) is exactly the kind of small, easy-to-miss detail that trips people up the first time they combine `RunnableParallel` and `.with_structured_output()` together.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)